ASSESSMENT

Notebook URL

https://github.com/MdAshraf8601/DE-Assessment/tree/main

This project implements a **data engineering pipeline using Databricks and PySpark** based on the **Medallion Architecture (Bronze, Silver, Gold)**.

The pipeline processes healthcare data from a FHIR (Fast Healthcare Interoperability Resources) API and transforms it into structured, analysis-ready datasets.

The data is sourced from the public FHIR API:

```
https://hapi.fhir.org/baseR4
```

Resources used:

- Patient
- Encounter
- Observation
- Condition

Due to network restrictions in the Databricks environment, the data was downloaded as JSON files and uploaded into **Databricks Volumes** for processing.

The project follows a **Medallion Architecture**:

```
Raw JSON → Bronze → Silver → Gold → SCD Dimension
```

Bronze Layer

- Stores raw ingested data in Delta format
- Minimal transformation applied
- Includes metadata such as ingestion timestamp and source

Silver Layer

- Cleans and transforms the data
- Flattens nested FHIR JSON structures
- Selects relevant fields
- Removes duplicates

□ Gold Layer

- Provides business-level aggregated insights

- Combines multiple datasets using joins
- Supports analytical queries

Data Processing Steps

1. **Data Ingestion**
    - JSON files loaded from Databricks Volume
    - FHIR Bundle structure parsed
2. **Data Transformation**
    - Nested arrays (`entry`) exploded
    - `resource` fields extracted
3. **Bronze Layer Creation**
    - Raw structured data stored in Delta tables
4. **Silver Layer Creation**
    - Cleaned and flattened data
    - Key attributes selected
5. **Multi-Resource Integration**
    - Patient, Encounter, Observation, and Condition datasets processed
6. **Gold Layer Creation**
    - Joined datasets to generate analytics
    - Created summary tables (e.g., patient-level metrics)

SCD Type 2 Implementation

A Slowly Changing Dimension (SCD Type 2) approach was implemented for the **Patient dimension**:

- Tracks historical changes in patient data
- Maintains:
    - `start_date`
    - `end_date`
    - `is_current` flag

This ensures that historical records are preserved instead of overwritten.

Key Features

- Handling of nested JSON (FHIR format)
- Implementation of Medallion Architecture
- Multi-table data integration
- SCD Type 2 for historical tracking
- Use of Delta Lake for storage
- Scalable and modular pipeline design

Final Output

The pipeline produces analytical datasets such as:

- Patient summary with:
    - Total encounters

- Total observations
- Total conditions

These outputs enable meaningful healthcare insights.

Technologies Used

- Databricks
- PySpark
- Delta Lake

Step 1: Data Ingestion

Load FHIR JSON data into Databricks

- Downloaded data from:

`https://hapi.fhir.org/baseR4/Patient?_count=50`

- Uploaded to:

`/Volumes/assessment/default/assessment/Dataset.json`

Code:

```
df = spark.read.option("multiline",
"true").json("/Volumes/assessment/default/assessment/Dataset.json")

display(df)
df.printSchema()
```

Step 2: Flatten JSON (FHIR Structure)

Extract actual patient data

Code:

```
from pyspark.sql.functions import explode

df_exploded = df.select(explode("entry").alias("entry"))

df_patient = df_exploded.select("entry.resource.*")

display(df_patient)
```

Step 3: Bronze Layer

Store raw structured data

Code:

```
from pyspark.sql.functions import current_timestamp, lit

df_bronze = df_patient.withColumn("extraction_timestamp", current_timestamp()) \
                      .withColumn("api_source", lit("FHIR_API"))

df_bronze.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("bronze_patient")
```

```
display(spark.table("bronze_patient"))
```

```
display(spark.table("bronze_patient"))
> See performance (1)
```

Table +

	birthDate	gender	id	meta	name	resourceType	text
1	1980-01-02	female	90235273	{"lastUpdated":"2026-02-09T18:33:41.627+00:00","source":"#yF6r58VrBxmshCYh","versionId":"...	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
2	1980-01-02	female	90236088	{"lastUpdated":"2026-02-09T18:33:46.634+00:00","source":"#rIQXCIS3xkct2WId","versionId":"1"}	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
3	1980-01-02	female	90239215	{"lastUpdated":"2026-02-09T18:34:01.640+00:00","source":"#NmeailrcflUsVeIs","versionId":"1"}	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
4	1980-01-02	female	90240275	{"lastUpdated":"2026-02-09T18:34:06.636+00:00","source":"#xDQAagxNTVkLvDAw","versionId...	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
5	1980-01-02	female	90242074	{"lastUpdated":"2026-02-09T18:34:16.648+00:00","source":"#rZhjWR9BmYN7mDbj","versionId...	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
6	1980-01-02	female	90242971	{"lastUpdated":"2026-02-09T18:34:21.646+00:00","source":"#cosXMpmxanyVVn6A","versionId...	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
7	1980-01-02	female	90243878	{"lastUpdated":"2026-02-09T18:34:26.647+00:00","source":"#LwD11wqY0sA2dyeU","versionId"...	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
8	1980-01-02	female	90244773	{"lastUpdated":"2026-02-09T18:34:31.639+00:00","source":"#SivjFc8PKG8g55d4","versionId":"1"}	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
9	1980-01-02	female	90245938	{"lastUpdated":"2026-02-09T18:34:36.645+00:00","source":"#W5vhPZy6Cv9QFL04","versionId"...	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
10	1980-01-02	female	90247103	{"lastUpdated":"2026-02-09T18:34:41.640+00:00","source":"#G4VYSctpGtGLIIGB","versionId":"...	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
11	1980-01-02	female	90248066	{"lastUpdated":"2026-02-09T18:34:46.705+00:00","source":"#a6X6CSs5CRiU91oq","versionId":...	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
12	1980-01-02	female	90249178	{"lastUpdated":"2026-02-09T18:34:51.647+00:00","source":"#GIA8TSeySBB1jXws","versionId":"...	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
13	1980-01-02	female	90250303	{"lastUpdated":"2026-02-09T18:34:56.673+00:00","source":"#qKtFdODOMQf3BtBh","versionId...	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":
14	1980-01-02	female	90251352	{"lastUpdated":"2026-02-09T18:35:01.643+00:00","source":"#xhIfuQxezPOPWFxA","versionId":...	[{"family":"Karla","given":["NuÃ±...	Patient	{"div":

All rows

Step 4: Silver Layer

Clean and prepare data

Code:

```
from pyspark.sql.functions import col

df = spark.table("bronze_patient")

df_silver = df.select(
    col("id"),
    col("gender"),
    col("birthDate"),
    col("meta.lastUpdated").alias("last_updated")
```

```
).dropDuplicates(["id"])

df_silver.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("silver_patient")

display(spark.table("silver_patient"))
```

```
display(spark.table("silver_patient"))
> See performance (1)
```

Table +

	id	gender	birthDate	last_updated
1	90254128	female	1980-01-02	2026-02-09T18:35:16.637+00:...
2	90263196	female	1980-01-02	2026-02-09T18:36:01.650+00:...
3	90282905	female	1980-01-02	2026-02-09T18:37:46.652+00:...
4	90286136	female	1980-01-02	2026-02-09T18:38:01.664+00:...
5	90289363	female	1980-01-02	2026-02-09T18:38:21.667+00:...
6	90268368	female	1980-01-02	2026-02-09T18:36:31.650+00:...
7	90269459	female	1980-01-02	2026-02-09T18:36:36.647+00:...
8	90279832	female	1980-01-02	2026-02-09T18:37:31.666+00:...
9	90280874	female	1980-01-02	2026-02-09T18:37:36.662+00:...
10	90286814	female	1980-01-02	2026-02-09T18:38:06.649+00:...
11	90258979	female	1980-01-02	2026-02-09T18:35:41.648+00:...
12	90245938	female	1980-01-02	2026-02-09T18:34:36.645+00:...
13	90256829	female	1980-01-02	2026-02-09T18:35:31.637+00:...
14	90291138	female	1980-01-02	2026-02-09T18:38:31.656+00:...
15	90235273	female	1980-01-02	2026-02-09T18:33:41.627+00:...

50 rows | 1.29s runtime

Step 5: SCD Type 2 (Patient Dimension)

Track history of patient data

Code:

```
from pyspark.sql.functions import current_date, lit

df_scd = spark.table("silver_patient") \
    .withColumn("start_date", current_date()) \
    .withColumn("end_date", lit(None).cast("date")) \
    .withColumn("is_current", lit(True))
```

```
df_scd.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("dim_patient")

display(spark.table("dim_patient"))
```

```
display(spark.table("dim_patient"))
> See performance (1)
```

Table +

	id	gender	birthDate	last_updated	start_date	end_date	is_current
1	90254128	female	1980-01-02	2026-02-09T18:35:16.637+00:...	2026-03-26	null	true
2	90263196	female	1980-01-02	2026-02-09T18:36:01.650+00:...	2026-03-26	null	true
3	90282905	female	1980-01-02	2026-02-09T18:37:46.652+00:...	2026-03-26	null	true
4	90286136	female	1980-01-02	2026-02-09T18:38:01.664+00:...	2026-03-26	null	true
5	90289363	female	1980-01-02	2026-02-09T18:38:21.667+00:...	2026-03-26	null	true
6	90268368	female	1980-01-02	2026-02-09T18:36:31.650+00:...	2026-03-26	null	true
7	90269459	female	1980-01-02	2026-02-09T18:36:36.647+00:...	2026-03-26	null	true
8	90279832	female	1980-01-02	2026-02-09T18:37:31.666+00:...	2026-03-26	null	true
9	90280874	female	1980-01-02	2026-02-09T18:37:36.662+00:...	2026-03-26	null	true
10	90286814	female	1980-01-02	2026-02-09T18:38:06.649+00:...	2026-03-26	null	true
11	90258979	female	1980-01-02	2026-02-09T18:35:41.648+00:...	2026-03-26	null	true
12	90245938	female	1980-01-02	2026-02-09T18:34:36.645+00:...	2026-03-26	null	true
13	90256829	female	1980-01-02	2026-02-09T18:35:31.637+00:...	2026-03-26	null	true
14	90291138	female	1980-01-02	2026-02-09T18:38:31.656+00:...	2026-03-26	null	true
15	90235273	male	1980-01-02	2026-02-09T18:33:41.627+00:...	2026-03-26	null	true

100 rows | 1.17s runtime

Step 6: Encounter Data Processing

Load Encounter JSON and Flatten

```
df = spark.read.option("multiline", "true") \
    .json("/Volumes/assessment/default/assessment/encounter.json")

from pyspark.sql.functions import explode

df_exploded = df.select(explode("entry").alias("entry"))
df_encounter = df_exploded.select("entry.resource.*")

display(df_encounter)
```

```python
df = spark.read.option("multiline", "true") \
    .json("/Volumes/assessment/default/assessment/encounter.json")

from pyspark.sql.functions import explode

df_exploded = df.select(explode("entry").alias("entry"))
df_encounter = df_exploded.select("entry.resource.*")

display(df_encounter)
```

See performance (1)

df: pyspark.sql.connect.dataframe.DataFrame = [entry: array, id: string ... 4 more fields]
df_encounter: pyspark.sql.connect.dataframe.DataFrame = [id: string, identifier: array ... 4 more fields]
df_exploded: pyspark.sql.connect.dataframe.DataFrame = [entry: struct]

Table

	id	identifier	meta	period	resourceType	subject
1	89187963	[{"value":"fdb73065-01cd-43c5-acf5-0624479e226...	{"lastUpdated":"2026-02-09T17:01:00.338+00:00","source":"#rSb2maCLHnQZY7n7","versionId":"...	{"start":"2025-09-01T08:13:04.0810...	Encounter	{"display":"Jane
2	89188091	[{"value":"32294fcd-e050-4487-b3ea-575539241e...	{"lastUpdated":"2026-02-09T17:01:01.338+00:00","source":"#2G69ns8X4eCdlndY","versionId":"1"}	{"start":"2025-09-01T08:49:54.6820...	Encounter	{"display":"Bob
3	89188157	[{"value":"5b8ec78e-5094-4a57-8f23-38719c3f57f...	{"lastUpdated":"2026-02-09T17:01:02.337+00:00","source":"#zAw0i3DTOhzKA6fQ","versionId":"1"}	{"start":"2025-09-01T09:04:55.3530...	Encounter	{"display":"Bill B
4	89188314	[{"value":"739796a9-2632-4823-860e-374f0e3ff60...	{"lastUpdated":"2026-02-09T17:01:03.339+00:00","source":"#Rlb1kghxDbLxRqJ7","versionId":"1"}	{"start":"2025-09-01T09:50:46.2890...	Encounter	{"display":"Chris
5	89188495	[{"value":"27b5e448-c001-4399-b748-49e72f5d41...	{"lastUpdated":"2026-02-09T17:01:04.340+00:00","source":"#3IffJNyyUaFIlF1M","versionId":"1"}	{"start":"2025-09-01T13:00:00.5580...	Encounter	{"display":"Jack
6	89188595	[{"value":"4eb83840-0a1e-496f-80f0-75bc3abc0be...	{"lastUpdated":"2026-02-09T17:01:05.340+00:00","source":"#Ef7SnrgOx0FnAerN","versionId":"1"}	{"start":"2025-09-01T13:02:15.3510...	Encounter	{"display":"Taylo
7	89188761	[{"value":"5c1f39a8-47d0-4100-8bcd-27acc5f86b4...	{"lastUpdated":"2026-02-09T17:01:06.339+00:00","source":"#RqGlBLy48etDv0KO","versionId":"1"}	{"start":"2025-09-02T11:30:04.8840...	Encounter	{"display":"Jon S
8	89188929	[{"value":"2ca5242c-a234-4606-af0c-6a42d3aab0a...	{"lastUpdated":"2026-02-09T17:01:07.341+00:00","source":"#u8FV7XdobpN519aV","versionId":"...	{"start":"2025-09-02T13:38:57.9990...	Encounter	{"display":"Geof
9	89189017	[{"value":"aa56cb5e018f4a2eb25a6244194816a6"}]	{"lastUpdated":"2026-02-09T17:01:08.339+00:00","source":"#8Gi1JIe2NGzRQnHG","versionId":"1"}	{"start":"2025-09-12T20:42:24.1687...	Encounter	{"display":"Jane
10	89189156	[{"value":"a0059382b0ed443caa5d91179da3e8ea"}]	{"lastUpdated":"2026-02-09T17:01:09.342+00:00","source":"#AiVK54U1E07aBHxh","versionId":"1"}	{"start":"2025-09-12T20:42:24.4372...	Encounter	{"display":"Jane
11	89189291	[{"value":"e80a94c700dc4917a02bab8555af9ed7"}]	{"lastUpdated":"2026-02-09T17:01:10.370+00:00","source":"#Fyyv5hxrAKffGn8l","versionId":"1"}	{"start":"2025-09-12T20:42:24.6548...	Encounter	{"display":"Jane
12	89189383	[{"value":"049ef80b9de54a2b9daa2a253f6b03ac"}]	{"lastUpdated":"2026-02-09T17:01:11.338+00:00","source":"#KfdHXoDIE4CzPunY","versionId":"1"}	{"start":"2025-09-12T20:42:24.8710...	Encounter	{"display":"John
13	89189490	[{"value":"fef254188a9e4f569914af173d5ab7e4"}]	{"lastUpdated":"2026-02-09T17:01:12.338+00:00","source":"#z6z4JYYvRqLpIKYt","versionId":"1"}	{"start":"2025-09-12T20:42:25.0880...	Encounter	{"display":"Rita
14	89877019	[{"value":"a037a5d9-98a5-41f3-b405-0f5d2c7ce49...	{"lastUpdated":"2026-02-09T18:01:53.888+00:00","source":"#Dm4iPZQDclyLxlND","versionId":"1"}	{"start":"2025-09-01T06:50:10.5710...	Encounter	{"display":"Gree

Bronze

```python
from pyspark.sql.functions import current_timestamp, lit

df_encounter_bronze = df_encounter.withColumn("extraction_timestamp",
current_timestamp()) \
                                  .withColumn("api_source", lit("FHIR_API"))

df_encounter_bronze.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("bronze_encounter")

display(spark.table("bronze_encounter"))
```

```python
display(spark.table("bronze_encounter"))
```

See performance (1)

Table

	id	identifier	meta	period	resourceType	subject
1	89187963	[{"value":"fdb73065-01cd-43c5-acf5-0624479e226...	{"lastUpdated":"2026-02-09T17:01:00.338+00:00","source":"#rSb2maCLHnQZY7n7","versionId":"...	{"start":"2025-09-01T08:13:04.0810...	Encounter	{"display":"Jane
2	89188091	[{"value":"32294fcd-e050-4487-b3ea-575539241e...	{"lastUpdated":"2026-02-09T17:01:01.338+00:00","source":"#2G69ns8X4eCdlndY","versionId":"1"}	{"start":"2025-09-01T08:49:54.6820...	Encounter	{"display":"Bob
3	89188157	[{"value":"5b8ec78e-5094-4a57-8f23-38719c3f57f...	{"lastUpdated":"2026-02-09T17:01:02.337+00:00","source":"#zAw0i3DTOhzKA6fQ","versionId":"1"}	{"start":"2025-09-01T09:04:55.3530...	Encounter	{"display":"Bill B
4	89188314	[{"value":"739796a9-2632-4823-860e-374f0e3ff60...	{"lastUpdated":"2026-02-09T17:01:03.339+00:00","source":"#Rlb1kghxDbLxRqJ7","versionId":"1"}	{"start":"2025-09-01T09:50:46.2890...	Encounter	{"display":"Chris
5	89188495	[{"value":"27b5e448-c001-4399-b748-49e72f5d41...	{"lastUpdated":"2026-02-09T17:01:04.340+00:00","source":"#3IffJNyyUaFIlF1M","versionId":"1"}	{"start":"2025-09-01T13:00:00.5580...	Encounter	{"display":"Jack
6	89188595	[{"value":"4eb83840-0a1e-496f-80f0-75bc3abc0be...	{"lastUpdated":"2026-02-09T17:01:05.340+00:00","source":"#Ef7SnrgOx0FnAerN","versionId":"1"}	{"start":"2025-09-01T13:02:15.3510...	Encounter	{"display":"Taylo
7	89188761	[{"value":"5c1f39a8-47d0-4100-8bcd-27acc5f86b4...	{"lastUpdated":"2026-02-09T17:01:06.339+00:00","source":"#RqGlBLy48etDv0KO","versionId":"1"}	{"start":"2025-09-02T11:30:04.8840...	Encounter	{"display":"Jon
8	89188929	[{"value":"2ca5242c-a234-4606-af0c-6a42d3aab0a...	{"lastUpdated":"2026-02-09T17:01:07.341+00:00","source":"#u8FV7XdobpN519aV","versionId":"...	{"start":"2025-09-02T13:38:57.9990...	Encounter	{"display":"Geo
9	89189017	[{"value":"aa56cb5e018f4a2eb25a6244194816a6"}]	{"lastUpdated":"2026-02-09T17:01:08.339+00:00","source":"#8Gi1JIe2NGzRQnHG","versionId":"1"}	{"start":"2025-09-12T20:42:24.1687...	Encounter	{"display":"Jane
10	89189156	[{"value":"a0059382b0ed443caa5d91179da3e8ea"}]	{"lastUpdated":"2026-02-09T17:01:09.342+00:00","source":"#AiVK54U1E07aBHxh","versionId":"1"}	{"start":"2025-09-12T20:42:24.4372...	Encounter	{"display":"Jane
11	89189291	[{"value":"e80a94c700dc4917a02bab8555af9ed7"}]	{"lastUpdated":"2026-02-09T17:01:10.370+00:00","source":"#Fyyv5hxrAKffGn8l","versionId":"1"}	{"start":"2025-09-12T20:42:24.6548...	Encounter	{"display":"Jane
12	89189383	[{"value":"049ef80b9de54a2b9daa2a253f6b03ac"}]	{"lastUpdated":"2026-02-09T17:01:11.338+00:00","source":"#KfdHXoDIE4CzPunY","versionId":"1"}	{"start":"2025-09-12T20:42:24.8710...	Encounter	{"display":"John
13	89189490	[{"value":"fef254188a9e4f569914af173d5ab7e4"}]	{"lastUpdated":"2026-02-09T17:01:12.338+00:00","source":"#z6z4JYYvRqLpIKYt","versionId":"1"}	{"start":"2025-09-12T20:42:25.0880...	Encounter	{"display":"Rita
14	89877019	[{"value":"a037a5d9-98a5-41f3-b405-0f5d2c7ce49...	{"lastUpdated":"2026-02-09T18:01:53.888+00:00","source":"#Dm4iPZQDclyLxlND","versionId":"1"}	{"start":"2025-09-01T06:50:10.5710...	Encounter	{"display":"Gree

50 rows | 1.09s runtime

Refreshed now

Silver (Final Correct Version)

```
from pyspark.sql.functions import col

df = spark.table("bronze_encounter")

df_silver = df.select(
    col("id"),
    col("subject").alias("patient_id")
).dropDuplicates(["id"])

df_silver.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("silver_encounter")
```

```
from pyspark.sql.functions import col

df = spark.table("bronze_encounter")

df_silver = df.select(
    col("id"),
    col("subject.display").alias("patient_id"),  # or reference later
    col("period.start").alias("start_time")
)

display(df_silver)
```

See performance (1)

df: pyspark.sql.connect.dataframe.DataFrame = [id: string, identifier: array ... 6 more fields]

df_silver: pyspark.sql.connect.dataframe.DataFrame = [id: string, patient_id: string ... 1 more field]

Table +

	id	patient_id	start_time
3	89188157	Bill Burr	2025-09-01T09:04:55.35300...
4	89188314	Christina P.	2025-09-01T09:50:46.28900...
5	89188495	Jack Hilton	2025-09-01T13:00:00.55800...
6	89188595	Taylor Downs	2025-09-01T13:02:15.35100...
7	89188761	Jon Snow	2025-09-02T11:30:04.88400...
8	89188929	Geoffrey Oyu	2025-09-02T13:38:57.99900...
9	89189017	Jane Doe	2025-09-12T20:42:24.16878...
10	89189156	Jane Doe	2025-09-12T20:42:24.43728...
11	89189291	Jane Doe	2025-09-12T20:42:24.65487...
12	89189383	John Doe	2025-09-12T20:42:24.87102...
13	89189490	Rita Zágoni	2025-09-12T20:42:25.08800...
14	89877019	Green Acres	2025-09-01T06:50:10.57100...
15	89877244	Pius Kariuki	2025-09-01T08:07:36.05100...
16	89877455	Jane Doe	2025-09-01T08:13:04.08100...

50 rows | 1.38s runtime

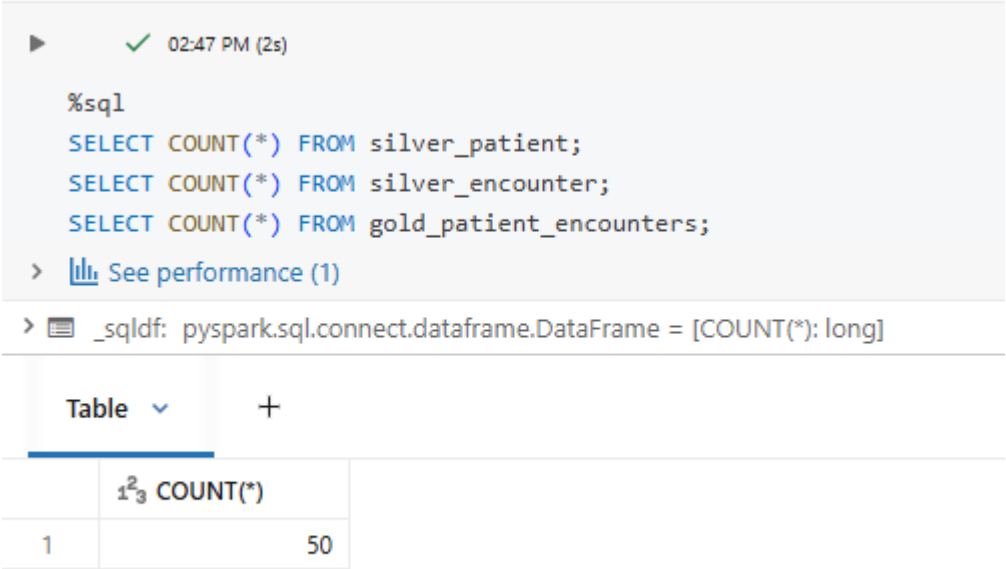
Gold Layer (Final Analytics)

```
%sql
CREATE OR REPLACE TABLE gold_patient_encounters AS
SELECT
    p.id AS patient_id,
    p.gender,
    COUNT(e.id) AS total_encounters
FROM silver_patient p
LEFT JOIN silver_encounter e
ON p.id = e.patient_id
GROUP BY p.id, p.gender;
```

Validation

```
SELECT COUNT(*) FROM silver_patient;
SELECT COUNT(*) FROM silver_encounter;
SELECT COUNT(*) FROM gold_patient_encounters;
```

Expected: **50 records**



```python
df = spark.read.option("multiline", "true") \
    .json("/Volumes/assessment/default/assessment/observer.json")

display(df)
df.printSchema()
```

> See performance (1)

> df: pyspark.sql.connect.dataframe.DataFrame = [entry: array, id: string ... 4 more fields]

Table +

	entry	id	link
1	> [{"fullUrl":"https://hapi.fhir.org/baseR4/Observation/95922264","resource":{"category":null,"code":{"coding":[{"code":"15074-8","display...	e4a38578-b8f5-401f-9dfd-f3de820be2a7	> [{"relation":"self","url":"https://hapi.fhir.org/baseR4/Observation?_count=50"},{"rel

```python
from pyspark.sql.functions import explode

df_exploded = df.select(explode("entry").alias("entry"))
df_observer = df_exploded.select("entry.resource.*")

display(df_observer)
```

> See performance (1)

> df_exploded: pyspark.sql.connect.dataframe.DataFrame = [entry: struct]

> df_observer: pyspark.sql.connect.dataframe.DataFrame = [category: array, code: struct ... 12 more fields]

```python
from pyspark.sql.functions import current_timestamp, lit

df_bronze = df_observer.withColumn("extraction_timestamp", current_timestamp()) \
                       .withColumn("api_source", lit("FHIR_API"))

df_bronze.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("bronze_observation")
```

> See performance (1)

```python
from pyspark.sql.functions import col, split

df = spark.table("bronze_observation")

df_silver = df.select(
    col("id"),
    col("status"),
    col("code.text").alias("observation_type"),
    split(col("subject.reference"), "/")[1].alias("patient_id")
)

df_silver.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("silver_observation")
```

See performance (1)

df: pyspark.sql.connect.dataframe.DataFrame = [category: array, code: struct ... 14 more fields]

df_silver: pyspark.sql.connect.dataframe.DataFrame = [id: string, status: string ... 2 more fields]

03:48 PM (1s) 47

```python
df = spark.read.option("multiline", "true") \
    .json("/Volumes/assessment/default/assessment/condition.json")
```

df: pyspark.sql.connect.dataframe.DataFrame = [entry: array, id: string ... 4 more fields]

03:48 PM (1s) 48

```python
from pyspark.sql.functions import explode

df_exploded = df.select(explode("entry").alias("entry"))
df_condition = df_exploded.select("entry.resource.*")

display(df_condition)
```

See performance (1)

df_condition: pyspark.sql.connect.dataframe.DataFrame = [abatementDateTime: string, asserter: struct ... 19 more fields]

df_exploded: pyspark.sql.connect.dataframe.DataFrame = [entry: struct]

```python
from pyspark.sql.functions import current_timestamp, lit

df_bronze = df_condition.withColumn("extraction_timestamp", current_timestamp()) \
                        .withColumn("api_source", lit("FHIR_API"))

df_bronze.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("bronze_condition")
```

See performance (1)

df_bronze: pyspark.sql.connect.dataframe.DataFrame = [abatementDateTime: string, asserter: struct ... 21 more fields]

```python
from pyspark.sql.functions import col

df = spark.table("bronze_condition")

df_silver = df.select(
    col("id"),
    col("clinicalStatus.text").alias("status"),
    col("code.text").alias("condition"),
    col("subject.display").alias("patient_id")
)

df_silver.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("silver_condition")
```

See performance (1)

df: pyspark.sql.connect.dataframe.DataFrame = [abatementDateTime: string, asserter: struct ... 21 more fields]

df_silver: pyspark.sql.connect.dataframe.DataFrame = [id: string, status: string ... 2 more fields]

```sql
%sql
CREATE OR REPLACE TABLE gold_patient_summary_full AS
SELECT
    p.id AS patient_id,
    p.gender,
    COUNT(DISTINCT e.id) AS total_encounters,
    COUNT(DISTINCT o.id) AS total_observations,
    COUNT(DISTINCT c.id) AS total_conditions
FROM silver_patient p
LEFT JOIN silver_encounter e ON p.id = e.patient_id
LEFT JOIN silver_observation o ON p.id = o.patient_id
LEFT JOIN silver_condition c ON p.id = c.patient_id
GROUP BY p.id, p.gender;
```

See performance (1)

```
%sql
CREATE OR REPLACE TABLE gold_patient_summary_full AS
SELECT
    p.id AS patient_id,
    p.gender,
    COUNT(DISTINCT e.id) AS total_encounters,
    COUNT(DISTINCT o.id) AS total_observations,
    COUNT(DISTINCT c.id) AS total_conditions
FROM silver_patient p
LEFT JOIN silver_encounter e ON p.id = e.patient_id
LEFT JOIN silver_observation o ON p.id = o.patient_id
LEFT JOIN silver_condition c ON p.id = c.patient_id
GROUP BY p.id, p.gender;
```

See performance (1)

Table +

num_affected_rows	num_inserted_rows

No rows returned

```
%sql
SELECT * FROM gold_patient_summary_full;
```

See performance (1)

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, gender: string ... 3 more fields]

Table +

	patient_id	gender	total_encounters	total_observations	total_conditions
1	90282034	female	0	0	0
2	90271469	female	0	0	0
3	90247103	female	0	0	0
4	90279832	female	0	0	0
5	90269459	female	0	0	0
6	90280874	female	0	0	0
7	90257875	female	0	0	0
8	90259920	female	0	0	0
9	90279367	female	0	0	0
10	90254128	female	0	0	0
11	90277570	female	0	0	0
12	90243878	female	0	0	0
13	90249178	female	0	0	0
14	90289363	female	0	0	0
15	90250303	female	0	0	0

50 rows | 1.77s runtime

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

FINAL ARCHITECTURE

FHIR JSON
↓
Bronze Layer
↓
Silver Layer
↓
Gold Layer
↓
SCD Dimension (dim_patient)

Summary

This project implements a data pipeline using Databricks and PySpark following the Medallion Architecture.
Data from FHIR API was ingested (via file upload due to network restrictions), processed into Bronze, Silver, and Gold layers, and analyzed using joins.
SCD Type 2 was implemented to maintain historical records of patient data.

This project demonstrates the design and implementation of a **scalable data pipeline** for healthcare data using modern data engineering practices. It highlights the ability to process semi-structured data, apply transformations, and generate actionable insights.